PROJECT SPECIFICATION

TaskFlow

Task Monitoring & Collaboration System

A complete functional requirements document for a role-based task management platform focused on assignment, collaboration, accountability, and progress visibility.

Project Goal

Build a secure, responsive and practical system where teams can create, assign, discuss, track and complete work from one centralized workspace.

This document defines the expected product behavior, user roles, workflows, features, validations, reporting, and completion criteria.

1. Project Overview

TaskFlow is a task monitoring and collaboration platform designed to help teams organize work, assign responsibilities, manage deadlines, share files, communicate around tasks, and monitor progress from a centralized dashboard.

The system should support multiple user roles and enforce permissions at both the user interface and server level. Every important task action should be traceable so that users can understand who created, assigned, changed, or completed work.

Core Product Principle
TaskFlow should feel like a real workplace application, not only a collection of CRUD screens. The final system should connect task ownership, collaboration, progress tracking, notifications, history, and reporting into one consistent workflow.

2. Project Objectives

- Provide a simple and reliable way to create, assign, manage, and complete tasks.
- Ensure users can clearly understand what is assigned to them and what they have assigned to others.
- Implement role-based access so every user can perform only authorized actions.
- Keep task discussions, files, progress, deadlines, and history in one place.
- Provide dashboard visibility into workload, status, deadlines, and overdue work.
- Maintain an auditable history of important changes.
- Deliver a responsive, intuitive experience across desktop, tablet, and mobile screens.

3. Role-Based Access Control

The application must implement clear role-based permissions. Permissions must be enforced on the backend as well as reflected in the interface.

Capability	Admin	Manager	Team Member
View all users	Yes	Team scope	Limited
Create tasks	Yes	Yes	Optional / permission based
Assign tasks	Yes	Yes	No by default
Edit own created tasks	Yes	Yes	No
Update assigned task status/progress	Yes	Yes	Yes
Delete tasks	Yes	Own created tasks	No
View global dashboard	Yes	No	No
View team dashboard	Yes	Yes	No

Capability	Admin	Manager	Team Member
Manage roles / activation	Yes	No	No
Comment and upload files	Yes	Yes	Yes

3.1 Admin

- Manage users, roles, and account activation status.
- View and manage all tasks across the system.
- Assign work to any active user.
- Access organization-wide task statistics, workload information, and overdue tasks.
- Review activity history and resolve administrative issues.

3.2 Manager

- Create and assign tasks to permitted users.
- Manage tasks created by them.
- Monitor work assigned to their team.
- Track progress, deadlines, comments, and activity history.
- View team workload and task statistics.

3.3 Team Member

- View tasks assigned to them.
- Update permitted status and progress fields.
- Add comments and task updates.
- Upload and access task attachments.
- View their own task statistics and activity.

4. Authentication & User Management

4.1 Authentication

- User registration and login.
- Secure authenticated sessions using token-based authentication.
- Protected application routes and protected APIs.
- Logout and session invalidation behavior.
- Forgot password and reset password flow.
- Change password from profile settings.

4.2 User Profile

- Name, email, profile image, phone number, and designation.
- Profile editing with validation.
- Personal task statistics.
- Account status visibility where appropriate.

4.3 User Administration

- Search and filter users.
- Assign or change roles.
- Activate or deactivate accounts.
- Prevent deactivated users from signing in or receiving new assignments.

- View basic workload and task statistics for each user.

5. Task Creation & Assignment

Authorized users should be able to create tasks and assign them to active registered users. A task should contain enough information for the assignee to understand the required work without relying on external communication.

5.1 Required Task Fields

- Task title
- Detailed description
- Assigned by
- Assigned to
- Priority: Low, Medium, High, Critical
- Status: Pending, In Progress, On Hold, Completed, Cancelled
- Progress percentage from 0 to 100
- Start date
- Due date
- Tags or labels
- Attachments
- Created date and last updated date

5.2 Task Ownership Rules

- The task creator remains visible as the task owner/assigner.
- Assigned users may update only fields permitted for their role.
- Deleting or reassigning a task must be permission controlled.
- A user must not gain access to another restricted task by changing an ID in a URL or request.
- Reassignment should generate a history entry and notify the newly assigned user.

5.3 Status & Progress

The normal workflow is Pending -> In Progress -> Completed. On Hold and Cancelled support exceptional cases. The system should prevent invalid transitions where business rules require it.

- Progress can be updated from 0 to 100.
- When a task is marked Completed, progress should become 100%.
- If progress becomes 100%, the interface may suggest or automatically set Completed according to the chosen workflow.
- Completed tasks should retain their completion timestamp.

6. Collaboration Features

6.1 Comments & Discussion

- Users involved in a task can add comments.
- Each comment displays author, profile image, message, date, and time.
- Comments should appear chronologically with the newest activity clearly visible.
- Comment deletion or editing should follow permission rules.

6.2 File Attachments

- Upload task-related documents and images.

- Display file name, type, size, uploader, and upload time.
- Allow authorized users to view and download attachments.
- Validate allowed file types and maximum file size.
- Allow authorized deletion of attachments.

6.3 Activity History

Important task changes must create activity entries. The activity feed should make the task lifecycle understandable without inspecting raw database records.

- Task created
- Task assigned or reassigned
- Status changed
- Progress changed
- Priority changed
- Due date changed
- Comment added
- Attachment uploaded or removed
- Task completed, reopened, placed on hold, or cancelled

7. Dashboard & Monitoring

The dashboard should provide useful information immediately after login. The numbers shown must respect the current user role and scope.

7.1 Summary Metrics

- Total tasks
- Tasks assigned by me
- Tasks assigned to me
- Pending tasks
- In Progress tasks
- On Hold tasks
- Completed tasks
- Overdue tasks
- Tasks due soon

7.2 Visual Reporting

- Task status distribution
- Priority distribution
- Completion trend over time
- Upcoming deadlines
- User or team workload for authorized roles

7.3 Deadline Monitoring

A task is overdue when its due date is earlier than the current date and it is not completed or cancelled. Overdue tasks should be clearly highlighted and included in dashboard metrics.

Deadline Experience
Show useful wording such as "Due tomorrow", "Due in 3 days", or "Overdue by 2 days" instead of forcing users to interpret dates manually.

8. Task Discovery, Search & Views

8.1 Search

- Search by task title.
- Search by description or tag where practical.
- Search results must still respect access permissions.

8.2 Filters

- Status
- Priority
- Assigned user
- Task creator
- Start date range
- Due date range
- Overdue only
- Completed / active tasks

8.3 Sorting

- Due date
- Created date
- Last updated date
- Priority
- Status

8.4 Views

- List or table view for detailed task management.
- My Tasks view with tabs for Assigned To Me, Assigned By Me, Pending, In Progress, Completed, and Overdue.
- Optional Kanban view grouped by task status with controlled drag-and-drop transitions.

9. Notifications & Reminders

The system should contain an in-app notification center so important task events are not missed.

9.1 Notification Events

- A task is assigned or reassigned.
- A task status changes.
- Priority changes.
- A comment is added.
- An attachment is uploaded.
- A due date is changed.
- A task is approaching its deadline.

- A task becomes overdue.

9.2 Notification Behavior

- Unread notification count.
- Mark individual notification as read.
- Mark all notifications as read.
- Clicking a task notification should open the related task.
- Avoid duplicate notifications for the same event.

10. Additional Features

The following features improve the project beyond basic task CRUD and can be implemented after the core workflow is stable.

10.1 Task Checklist

- Add multiple checklist items inside a task.
- Mark individual items complete.
- Show checklist completion count and percentage.

10.2 Subtasks

- Create smaller units of work under a parent task.
- Allow individual assignees and due dates where appropriate.
- Show parent task progress based on subtask completion if implemented.

10.3 Task Dependencies

- Allow a task to be blocked by another task.
- Clearly show blocking and blocked tasks.
- Warn before completing a task whose required dependency is incomplete.

10.4 Recurring Tasks

- Support recurring work such as daily, weekly, or monthly tasks.
- Automatically create the next task instance according to the recurrence rule.

10.5 Mentions & Watchers

- Allow @mentions in comments.
- Allow users to follow/watch a task and receive relevant updates without being the assignee.

10.6 Saved Filters

- Allow users to save common views such as "High Priority Due This Week" or "My Overdue Tasks".

10.7 Export & Reporting

- Export filtered task data to CSV or spreadsheet format.
- Generate printable task or summary reports where required.

10.8 Real-Time Updates

- Update comments, status changes, and notifications without requiring a full page refresh.

10.9 Archive & Reopen

- Archive old completed tasks without deleting history.
- Allow authorized users to reopen completed tasks with an activity record.

11. Required Application Screens

1. Login and registration screens.
2. Forgot password and reset password screens.
3. Role-aware dashboard.
4. All Tasks / Task Listing page.
5. My Tasks page.
6. Create Task page or modal.
7. Task Details workspace.
8. Edit Task screen for authorized users.
9. Notification center.
10. User profile and password settings.
11. User Management page for Admin.
12. User / team workload view for authorized roles.

11.1 Task Details Workspace

- Task overview: title, description, status, priority, progress, dates, assigner, assignee.
- Attachments section.
- Comments/discussion section.
- Activity/history section.
- Quick actions based on user permissions.

12. Suggested Data Structure

The exact schema may be refined during implementation, but the system should clearly model the following entities and relationships.

Users

```
name, email, passwordHash, role, profileImage,
phone, designation, isActive, createdAt, updatedAt
```

Tasks

```
title, description, assignedBy, assignedTo, priority,
status, progress, startDate, dueDate, tags[], attachments[],
completedAt, createdAt, updatedAt
```

Comments

```
taskId, userId, message, createdAt, updatedAt
```

Activities

```
taskId, userId, action, field, oldValue, newValue, createdAt
```

Notifications

```
userId, taskId, type, message, isRead, createdAt
```

Optional Checklist / Subtasks

```
taskId or parentTaskId, title, assignedTo, status,
isCompleted, dueDate, createdAt, updatedAt
```

13. Functional API Areas

The backend should expose clean, consistent endpoints grouped by domain. Exact route naming can vary, but the application should support the following operations.

13.1 Authentication & Profile

- Register, login, logout, current profile, update profile, change password, request password reset, reset password.

13.2 Users

- List/search users, view user details, update role, activate/deactivate user, retrieve user task statistics.

13.3 Tasks

- Create, list, view details, edit, delete, assign/reassign, update status, update progress, archive/reopen.

13.4 Collaboration

- Add/list/edit/delete comments according to permissions.
- Upload/list/download/delete attachments according to permissions.
- Read task activity history.

13.5 Notifications & Dashboard

- List notifications, mark read, mark all read.
- Retrieve dashboard summary metrics, deadlines, charts, and workload data.

14. Validation, Security & Reliability Requirements

14.1 Validation

- Title and assignee are required for task creation.
- Due date cannot be earlier than start date.
- Progress must remain between 0 and 100.
- Role and status values must be validated against allowed values.
- File type and file size restrictions must be enforced server-side.
- Invalid IDs and missing records must return clear errors.

14.2 Authorization

- Every protected endpoint must verify authentication.
- Every restricted action must verify role and resource ownership.
- Do not rely only on hiding buttons in the frontend.
- Users must not be able to access another restricted task by manually changing request parameters.

14.3 Error Handling

- Use consistent success and error response structures.
- Return appropriate status codes for validation, authentication, authorization, not-found, and server errors.
- Show user-friendly error messages in the interface.

- Log unexpected server errors without exposing sensitive implementation details to users.

14.4 Data Integrity

- Do not assign tasks to inactive or nonexistent users.
- Deleting or deactivating a user must not silently remove historical task ownership.
- Important task changes should create corresponding activity records.
- Avoid duplicate notifications caused by repeated requests.

15. UI/UX Expectations

- Modern, clean, and professional visual design.
- Responsive layouts across desktop, tablet, and mobile.
- Clear status and priority badges.
- Consistent forms, spacing, typography, and button hierarchy.
- Useful empty states and loading skeletons.
- Confirmation dialogs for destructive actions.
- Toast or inline feedback for successful and failed actions.
- Pagination or infinite loading where task/user lists can grow large.
- Accessible form labels, focus states, keyboard navigation, and readable contrast.

16. Minimum Acceptance Criteria

The project should not be considered complete until the full task workflow works end to end.

1. A user can register/login and access only authorized screens and APIs.
2. Admin, Manager, and Team Member permissions behave correctly.
3. Authorized users can create, assign, edit, and manage tasks.
4. Assigned users can view their work and update permitted status/progress fields.
5. Comments and attachments work from the task details workspace.
6. Activity history records important task changes.
7. Dashboard statistics reflect the current user role and actual task data.
8. Search, filters, sorting, and pagination work together correctly.
9. Notifications are created for the defined task events and can be marked read.
10. Overdue and upcoming deadline calculations are correct.
11. Frontend and backend validation prevent invalid data and unauthorized actions.
12. The interface is responsive and usable on mobile as well as desktop.
13. The application has no major broken flows, console errors, or unhandled server errors.

17. Final Deliverables

- Complete frontend source code.
- Complete backend source code.
- Database models and relationships.
- Working authentication and role-based authorization.
- Complete task workflow with collaboration and monitoring features.
- Environment configuration example without real secrets.
- README with setup and run instructions.

- API documentation or endpoint reference.
- Test/demo accounts for each role.
- Deployed working application or deployment-ready configuration.
- Clean Git repository with meaningful commit history.

18. Final Expected Workflow

End-to-End Flow

Login -> Dashboard -> Create Task -> Select Assignee -> Set Priority & Deadline -> Assign Task -> Assignee Receives Notification -> Open Task -> Start Work -> Update Progress -> Add Comments / Attachments -> Activity History Updates -> Complete Task -> Dashboard & Reports Update

19. Final Product Expectation

The finished TaskFlow application should function as a complete task monitoring and collaboration product rather than a basic CRUD project. Authentication, authorization, assignment, collaboration, progress tracking, notifications, history, deadline monitoring, dashboard reporting, and responsive design should work together as one connected system.

The implementation should prioritize correctness, security, maintainability, clear permissions, and a polished user experience.